

**PONTIFICIA UNIVERSIDAD CATOLICA DEL PERU
UNIVERSIDAD PERUANA CAYETANO HEREDIA**

**FACULTAD DE CIENCIAS E INGENIERÍA
INGENIERÍA BIOMÉDICA**



TALLER N°7: Informe Técnico Aplicación: Interfaz de sensor flex con ESP32 con MIT App Inventor

Autores:

Arana Ocros, Christian Jesus
Auqui Jota, Luciano Mateo Alejandro
Bastidas Reyes, Mariangel Beatriz
Bravo Larico, Juan Pablo Rai
Crisologo Ramirez, Renato Del Piero
Huidobro Kokubun, Miranda Naomi

Asesores:

Zuñiga Mamani, Juan Manuel
Venancio, Julissa
Nuñez, Isabel
Med. Lazarte, Rodney

Curso:

Fundamentos de Biodiseño

2026

Conexión Hardware

El circuito fue implementado con el uso de un divisor de tensión conformado por un sensor flex de 2.2" Spectra Symbol en serie con una resistencia de 47k Ω . El nodo superior del sensor se conectó al pin 3.3V del ESP32. El nodo intermedio se conectó al pin analógico GPIO 36 del ESP32. Finalmente, el nodo que se encuentra conectado al extremo libre de la resistencia se conectó al GND.

Calibración y rango obtenido

Condición/ángulo conocido (°)	ADC	Tensión (mV)	Valor mapeado
0	951	766	0
30	1339	1079	27
45	1652	1331	49
60	1839	1481	62
90	2435	1962	90

La calibración se realizó midiendo los valores crudos del ADC y obteniendo los valores obtenidos respectivamente de milivoltios y grados de flexión mediante mapeo lineal con ayuda del código.

Decisiones de diseño

Se implementó el uso de la librería WifiManager para la configuración de las credenciales de red. A continuación, se configuró una ruta específica para que el ESP32 responda con el valor numérico en texto plano, de esta manera, se facilita el procesamiento para la aplicación móvil. Además, se implementó la función constrain para limitar los valores mapeados al rango estimado mediante experimentación.

Principales Errores Encontrados

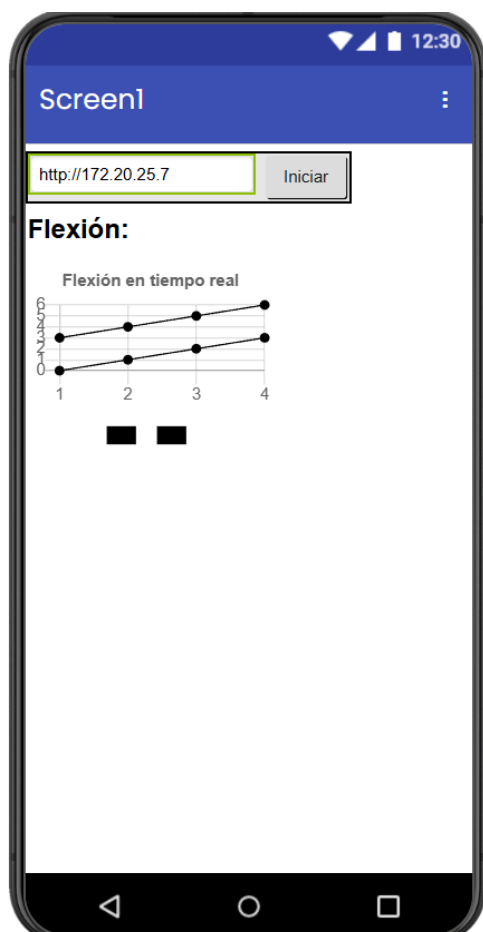
En primera instancia, existió un conflicto con el programa en el que no se detectaba correctamente el ESP32 a través del COM correspondiente de la laptop. Esto se producía debido al uso de una laptop carente de los drivers necesarios para detectarla. Se solucionó este problema con el uso de tutoriales en internet y, posteriormente, el uso de las computadoras prestadas por la universidad.

Después, ocurrieron problemas con distintos dispositivos del circuito. Se empleó un cable microUSB que únicamente transmite corriente, no datos, imposibilitando la detección por parte de la laptop. Además, por temas de tiempo se planteó la posibilidad de utilizar material prestado de un compañero. Sin embargo, por el mal uso del sensor flex, el cuál es uno

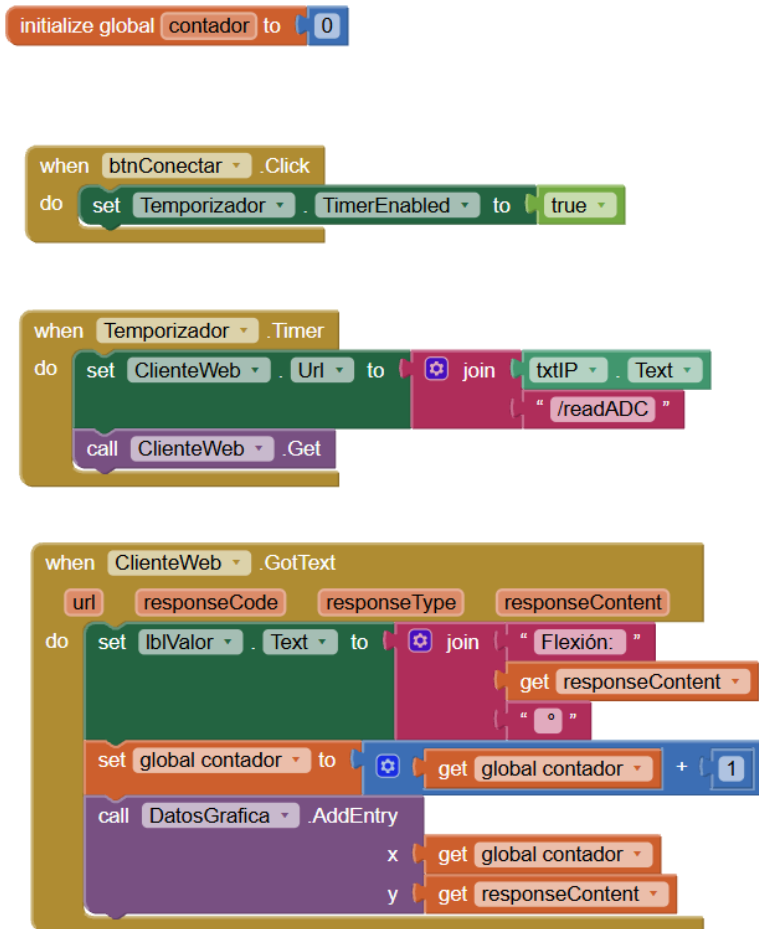
delicado, se notó la falta de resistencia del dispositivo con ayuda de un multímetro, lo cuál indicaba que el sensor se había malogrado.

Finalmente, se pudieron solucionar todos estos problemas en la última sesión, en la que solo se destacan como errores ciertas fallas mínimas como pequeños errores en el código para la obtención de la dirección IP del sensor y unas correcciones de ultimo momento con la programación de bloques.

Se producían varias instancias en las que se registraba únicamente valores de 0mV y voltajes erráticos. Esto era provocado por una referencia de GND flotantes y un falso contacto inicial en el protoboard. Se solucionó haciendo presión de tal manera que se aseguró el puente analógico entre el GND del ESP32 y el nodo perteneciente al extremo libre de la resistencia.



Captura del programa MIT App Inventor: Interfaz



Captura del programa MIT App Inventor: Programación por bloques

```
#include <WiFiManager.h>
#include <WebServer.h>

const int FLEX_PIN = 36;    // GPIO36 (ADC1)

WebServer server(80);

// AJUSTAR con la calibración real del equipo
int mv_min = 760;    // ejemplo: posición recta
int mv_max = 1791;    // ejemplo: posición flexionada

void handleADC() {
    int raw = analogRead(FLEX_PIN);    // 0-4095
    int mv = map(raw, 0, 4095, 0, 3300);    // mV

    // Si el ángulo queda invertido, intercambie mv_min y mv_max
    int angle = map(mv, mv_min, mv_max, 0, 90);
    angle = constrain(angle, 0, 90);
}
```

```

Serial.print("ADC: ");
Serial.print(raw);
Serial.print(" | mV: ");
Serial.print(mv);
Serial.print(" | Angulo: ");
Serial.println(angle);

// En esta versión se envía el ángulo como texto simple.
// Si se desea visualizar mV, reemplace String(angle) por String(mv).
server.send(200, "text/plain", String(angle));
}

void setup() {
  Serial.begin(115200);

  WiFiManager wm;

  bool conectado = wm.autoConnect("FlexAP", "12345678");

  if (!conectado) {
    Serial.println("No se pudo conectar al WiFi");
    ESP.restart();
  }

  Serial.println("WiFi conectado");
  Serial.print("IP del ESP32: ");
  Serial.println(WiFi.localIP());

  server.on("/readADC", HTTP_GET, handleADC);

  server.begin();

  Serial.println("Servidor iniciado");
}

void loop() {
  server.handleClient();
}

```

Código .ino empleado



Captura de aplicación móvil

[Video demostración](#)